

6-8-2010

Isolates: Serializability Enforcement for Concurrent ML

Lukasz Ziarek

Purdue University, lziarek@cs.purdue.edu

Armand Navabi

Purdue University, Anavabi@cs.purdue.edu

Suresh Jagannathan

Purdue University, suresh@cs.purdue.edu

Report Number:

10-007

Recommended Citation

Ziarek, Lukasz; Navabi, Armand; and Jagannathan, Suresh, "Isolates: Serializability Enforcement for Concurrent ML" (2010). *Department of Computer Science Technical Reports*. Paper 1731.
<https://docs.lib.purdue.edu/cstech/1731>

Isolates: Serializability Enforcement for Concurrent ML

Lukasz Ziarek, Armand Navabi, and Suresh Jagannathan

Purdue University
{lziarek, anavabi, suresh}@cs.purdue.edu

Abstract. There has been much recent interest in exploring higher-level concurrency control abstractions such as software transactional memory (STM) to alleviate the complexity of reasoning about interactions among concurrent threads of control. *Isolation* and *atomicity* are the two critical properties provided by an STM that guarantee serializability of concurrent actions. Isolation ensures that transactions execute without interference from effects performed by other transactions, and atomicity guarantees that intermediate effects performed by a transaction are not seen by other concurrently executing transactions.

While these properties have been primarily designed with shared memory in mind, there has been recent work (5; 6) that explores how atomicity could be leveraged to increase the expressivity of message-passing abstractions such as the first-class synchronous events found in Concurrent ML (CML) (17). Notably, these proposals do *not* enforce isolation of concurrently executing events, and thus cannot be used to enforce transactional execution of CML programs. In this paper, we consider the introduction of a new event combinator that addresses this significant limitation. An *isolate* is a combinator that allows a complex event to execute in isolation with other concurrently executing events (including other isolates). By doing so, it enables the integration of a true transactional semantics into a CML-style concurrency model, enabling reasoning about CML programs in terms of serializable event orderings. Incorporating isolation into CML poses a number of challenging problems, however, whose solutions form the focus of this paper.

1 Introduction

Programming with concurrency is challenging because reasoning about non-deterministic interactions among concurrently executing computations is difficult. Concurrency control abstractions such as software transactions (15) have attracted significant interest because they simplify reasoning about these interactions. Transactions provide two key guarantees on the computations they encapsulate: (1) *isolation* ensures that a computation can execute without interference from effects performed by other threads; (2) *atomicity* ensures that intermediate effects performed within a transaction do not become visible until the transaction completes.

Recently, *transactional events* (5; 6) have been proposed as a way to leverage the power of atomicity to increase the expressivity of message-passing abstractions such as first-synchronous events found in languages like Concurrent ML (CML) (17). While CML allows the construction of complex events from base events, synchronization can only take place on a single event. This limitation makes it difficult to express certain communication protocols and impossible to express others, such as three-way rendezvous. Transactional events address this limitation by allowing multiple communication actions to be encapsulated within a new event combinator (`thenEvt`) that makes the effects of these actions visible to other threads provided all of them can succeed. For example, the expression, `thenEvt(sendEvt (c1, v), fn() => recvEvt c2)`, when synchronized will only complete if the operation executes in a state in which both the `sendEvt` and `recvEvt` can succeed in that order as a single atomic action. The guarantee of atomicity provided by `thenEvt` distinguishes it from the functionality provided by other CML event combinators, and leads to a substantial increase in expressive power and programmability.

Unfortunately, unlike mainstream software transactions, two threads concurrently executing `thenEvs` can have their communication effects witnessed by the other; when this happens, a larger atomic unit is formed. Thus, while transactional events provide an all-or-nothing property on the events they encapsulate, they provide no mechanism to *restrict* visibility of effects performed within the dynamic context of a transactional event. Repairing this limitation forms the focus of this paper. The ability to isolate effects of different concurrently executing transactional events simplifies program reasoning, and prevents unwanted interactions. Without isolation, any effect witnessed by one transactional event performed by another effectively pairs the two together: even though failure of one results in the failure of the other, there is no facility available to prevent these intermediate actions from being observed in the first place.

To support isolation, we introduce *isolates*, a new abstraction that enforces isolation of concurrently evaluating events. The ability to execute concurrent events in isolation along with the all-or-nothing atomicity property of transactional events not only allows the construction of truly transactional (i.e., serializable) CML programs, but leads to additional expressivity and efficiency that would otherwise not be possible.

The remainder of the paper is organized as follows. The next section briefly describes CML and transactional events. Section 3 provides a motivating example. Section 4 presents a naive semantics for isolates that imposes a simple *a priori* ordering on isolate evaluation that ensures serializability but at the price of constraining concurrency. In Section 5 we relax this restriction; our new formulation uses *capabilities* to track potential interfering communication actions, limiting concurrent evaluation only when necessary. Section 6 states soundness results that relate the two different formulations. Section 7 discusses implementation issues, and Section 8 presents related work and conclusions.

2 CML and Transactional Events

Concurrent ML (17) (CML) is a concurrent extension of Standard ML that utilizes synchronous message passing for communication among concurrently executing threads. Threads perform `send` and `recv` operations on typed channels; these operations block until a matching action on the same channel is performed by another thread.

CML also provides first-class synchronous *events* that abstract synchronous message-passing operations. An event value of type `'a event` when synchronized yields a value of type `'a`. Thus, an event value represents a potential computation, with latent effect until a thread synchronizes upon it by calling `sync`. The following equivalences thus hold: `send(c, v) ≡ sync(sendEvt(c,v))` and `recv(c) ≡ sync(recvEvt(c))`. Besides `sendEvt` and `recvEvt`, there are two other base events that we refer to in the paper: `alwaysEvt` given a value returns an event that when synchronized upon returns that value; `neverEvt` yields an event that can never be successfully synchronized upon.

Much of CML's expressive power derives from event combinators that construct complex event values from other events. We list some of these combinators in Fig. 1. The `chooseEvt` event combinator takes a list of events and constructs an event value that represents the non-deterministic choice of the events in the list; for example, `chooseEvt[recvEvt(a), sendEvt(b,v)]` when synchronized will either receive a unit value from channel `a`, or send value `v` on channel `b`. The expression `wrap(ev, f)` creates an event that when synchronized applies the result of synchronizing on event `ev` to function `f`. Conversely, `guard(f)` creates an event which when synchronized evaluates `f()` to yield event `ev` and then acts as `ev`.

```

sendEvt : a chan * a -> unit event
recvEvt : a chan -> a event
neverEvt : a event
alwaysEvt : a -> a event
sync : a event -> a
choose : a event list -> a event
wrap : a event * (a -> b) -> b event
guard : (unit -> a event) -> a event

```

Fig. 1. CML event operators.

While CML's event combinators provide a great deal of expressivity, there are nonetheless useful abstractions that are difficult or impossible to define. One such example is a safe guarded receive; given a channel `c` and a guard `g`, the operation accepts `v` from `c` *only if* `g(v)` yields true. Expressing this functionality in CML is difficult because the act of transmitting a value from the sender to receiver requires a synchronization; once the synchronization is complete, there is no facility to revoke the communication in case the guard yields false.

To address these limitations, a *transactional event* combinator (written as `thenEvt(evt,f)`), that sequences multiple communication actions into a single atomic event is necessary (5; 6). A transactional event takes an event `evt` and a function `f` for producing a new event from the result of the first event. A synchronization action on a `thenEvt` first (provisionally) synchronizes on `evt`, and calls `f` with the resulting value. The event yielded by the application is then synchronized on as well. If both synchronizations succeed (i.e., neither event yields `neverEvt`) the transactional event succeeds, yielding the value of the last event; if either fails, the entire event fails to synchronize, effectively erasing any of the provisional actions. Using transactional events, we can now easily express a guarded receive:

```
thenEvt(recvEvt(ch),
        fn(v) => if g(v) then alwaysEvt(v)
                  else neverEvt)
```

Because the `neverEvt` returned when `g(v)` yields false can never be synchronized upon, we ensure that the act of receiving a value from channel `ch` occurs only if `g(v)` is true. Thus, transactional events abstractly define atomic sets of communication actions through the use of the `thenEvt` combinator. Such sets are dynamically linked to create larger atomic units.

3 The Need for Isolation

Consider a server abstraction that mediates access to a pool of identical resources. The server provides three operations: (1) *query* that returns information about the resources it holds; (2) *reserve* that requests some number of these resources to a client; and (3) *release* that returns a previously reserved set of resources back to the pool. A simple implementation of the server written in CML in which resources are abstracted as integers is given in Fig. 2.

The server is implemented as a thread that loops, repeatedly waiting for input on a dedicated input channel. The server loop is implemented as a transactional event that encapsulates the act of reading the next operation from a client, and yielding the result. If a client asks to reserve a number of resources fewer than the number of available, the server indicates the success of the operation by yielding an `alwaysEvt`; if the client request is not satisfiable, a `neverEvt`, which can never be successfully synchronized against, is returned, thus preventing a client communicating with the server from within its own transactional event from committing any other actions performed by that event. The `sync` operation is successful only if the requested operation is successful, in which case the new server state is available for the next iteration.

With this interface, we might consider writing a client (see Fig. 3 that queries a server to check the number of resources it has, and based on the result, makes a request to reserve some number of them, using transactional events to ensure that the query and reservation execute atomically).

The `query` function communicates to the server to query the server's state; once known, the client uses auxiliary function `f` to reserve some number of

```

datatype ops = Query of int chan | Req of int | Rel of int

fun server(n) =
  let val reqCh = channel()
  fun serverLoop(n) =
    let val evt =
      thenEvt(recvEvt(reqCh),
        fn (req) =>
          case req of
            Query ch => wrapEvt(sendEvt(ch,n),
                               fn () => alwaysEvt(n))
          | Res i => if n >= i
                    then alwaysEvt(n-i)
                    else neverEvt
          | Rel j => ...)
    in serverLoop(sync(evt))
    end
  in (spawn(serverLoop(n));
    reqCh)
  end

```

Fig. 2. A simple server abstraction that uses transactional events.

```

fun query(server,replyCh) =
  thenEvt(sendEvt(server,Query(replyCh))
    fn () => recvEvt(replyCh)

fun client(server,f) =
  let val replyCh = channel()
  val evt = thenEvt(query(server,replyCh),
    fn (n) => let val k = f(n)
              in sendEvt(server,Res(k))
              end
  in sync(evt)
  end

```

Fig. 3. A client that defines a protocol involving multiple communication events with a server.

resources based on this state. While this solution is disarmingly simple, it is unfortunately incorrect. This is because the transactional event that defines the server loop can commit only when the client's transaction does. But, the client requires *two* operations involving the server to be atomically executed, the first to perform the query, and the second to make the actual reservation. However, the server will not initiate the next iteration of its server loop until the query first performed by the client successfully commits; this transaction cannot commit until the client can successfully complete its second communication with the server. In other words, the communication parity mismatch between the server and client foils the construction of a complex atomic client-side protocol: the client requires two communications with the server for its transaction to be successful, whereas the server only expects a single interaction. Simply changing the client implementation so that the query and subsequent reservation do not execute within a transactional event would break obvious atomicity requirements. Alternatively, we could change the server code to accept two requests as part of its transactional event; this would allow the client protocol to succeed for this example, but would be a very brittle solution, since it not work for other kinds of protocols that initiate a different number of communication actions with the server.

An alternative is to modify the server by allowing it to accept an arbitrary number of requests as part of a given transaction (see Fig. 4).

```

fun server (n) =
  let val reqCh = channel()
      fun serverLoop(n) =
        let val evt =
          thenEvt(recvEvt(reqCh),
            fn (req) =>
              wrap (case req of
                Query ch =>
                  wrapEvt(sendEvt(ch,n)
                    fn () => alwaysEvt(n))
                | Res i => if n < i
                  then neverEvt
                  else alwaysEvt(n-i)
                | Rel j => ...),
            fn (n) => serverLoop(n))
        in sync(evt)
        end
      in (spawn(serverLoop(n));
        reqCh)
    end

```

Fig. 4. A server implementation that accepts multiple requests as part of a given transaction.

In this revised implementation, the internal server loop itself is implemented as a transactional event. This enables the server to receive multiple requests as part of a complex client protocol. Unfortunately, there remain problems with this solution as well. The simpler problem is that the loop itself does not have a termination condition, and thus has no apparent commit point; this can be easily fixed by extending the interface to allow clients to notify the server when the client-side transaction is complete.

A more problematic issue is the loss of serializability due to the possibility the server may accept requests from *different* clients while participating in an ongoing transaction. Consider clients C_1 and C_2 both implementing the protocol described above. Suppose both issue queries to the server and receive the same result indicating the current server state. If client C_1 now succeeds in performing its reservation, C_2 's subsequent computation of its desired reservation, which was based on a server state that is no longer accurate, is now incorrect. Although atomicity is preserved – none of the operations results in a `neverEvt` being yielded by the server, and thus all communication actions succeed – isolation is lost because C_1 's effects are visible in the middle of C_2 's transaction.

To provide a solution that permits multiple clients to concurrently communicate with the server without violating serializability guarantees, we introduce a new abstraction that *isolates* the execution of one transactional event from the concurrent effects performed by another. The abstraction is expressed using a new event combinator: `isolateEvt: 'a evt  $\rightarrow$  'a evt` that given an event value yields an isolated event value that when synchronized executes in isolation of all other isolated events.

Thus, to ensure that a client executes in isolation from all other clients, we could write:

```
fun client(server,f,dom) =
  let val replyCh = channel()
      val evt = client protocol
  in sync(isolateEvt(evt))
  end
```

To prevent interference induced by such communication, isolate evaluation ensures that all communication to the server by C_1 are performed prior to all communication by C_2 or *vice versa*, effectively serializing their execution with respect to their common channels. Because the communication actions performed by the event arguments to an `isolateEvt` may be arbitrarily complex, enforcing such ordering requires tracking the effects performed by the event arguments transitively; in the following sections, we discuss how to efficiently identify and collect this information.

4 Semantics

Our semantics is defined in terms of a core call-by-value functional language with threading and communication primitives. Communication between threads is achieved using synchronous channels and transaction-encapsulated events. Our

language extends a transactional event core language with one additional construct used to express isolated events. For perspicuity, the language omits many useful event combinators such as `chooseEvt`, `wrap`, or `guard` since they raise no interesting semantic issues with respect to isolates. References are also omitted for this reason.

We first present a semantics for this language whose syntax and grammar is shown in Fig. 5 using a naive definition of isolates in which concurrent evaluation of isolated events is not allowed; we subsequently consider refinements to this semantics that relax this restriction.

$$\begin{array}{ll}
e := \text{unit} \mid \gamma \mid x \mid \lambda x.e \mid e \ e & v := \text{unit} \mid c \mid \gamma \mid \lambda x.e \\
\mid \text{spawn } e \mid \text{sync } e \mid \text{ch}() & \mid \text{sendEvt}(v, v) \mid \text{recvEvt}(v) \\
\mid \text{sendEvt}(e, e) \mid \text{recvEvt}(e) & \mid \text{neverEvt} \mid \text{alwaysEvt } v \\
\mid \text{neverEvt} \mid \text{alwaysEvt } e & \mid \text{thenEvt } (v, v) \mid \text{isolateEvt}(v) \\
\mid \text{thenEvt } (e, e) \mid \text{isolateEvt}(e) & \\
\\
E := \cdot \mid E \ e \mid v \ E & \mathcal{F} := \cdot \mid \\
\mid \text{sync } E & \mid \text{thenEvt } (\mathcal{F}, v) \mid \\
\mid \text{sendEvt}(E, e) \mid \text{sendEvt}(c, E) & \mid \text{isolateEvt}(v) \mid \\
\mid \text{recvEvt}(E) \mid \text{alwaysEvt } E & \mid \text{alwaysEvt } v \\
\mid \text{thenEvt } (E, e) \mid \text{thenEvt } (v, E) & \\
\mid \text{isolateEvt}(E) & \\
\\
M \in \text{ContextStack} := \mathbf{I} \mid E \mid \mathcal{F} \mid M : M & \\
T \in \text{NonTransactionThread} := (\mathbf{t}, e) & \\
\overline{T} := T \mid T \parallel \overline{T} & \\
K \in \text{TransactionThread} := (\mathbf{t}, M, e) & \\
\overline{K} := K \mid K \parallel \overline{K} &
\end{array}$$

Fig. 5. Language Syntax and Grammar

In our syntax (see Fig. 5) v ranges over values, c over channel references, γ over constants, e over expressions, and $\mathbf{t}$ over thread identifiers. The semantics shown in Fig. 6 defines three relations. The $\hookrightarrow$ relation defines thread-local actions that can be performed within or outside a transactional context. Function application (rule APP) and channel creation (rule CHANNEL) can be performed in either context.

Global evaluation is defined via relation $\rightarrow$. A global state consists of a set of transactional threads ($\overline{K}$) and non-transactional threads ($\overline{T}$). Threads that get spawned (rule SPAWN) are initially not part of any ongoing transaction, and are initially added to the set of non-transactional threads ($\overline{T}$). The STEPTHREAD rule simply allows local execution within a non-transactional thread.

A thread gets added to $\overline{K}$ when it attempts to synchronize an event (rule SYNCTHREAD). Thus, *every* event synchronization initiates a transaction. For

APP	CHANNEL	SPAWN
$\frac{}{(\lambda x.e)v \hookrightarrow e[v/x]}$	$\frac{c \text{ fresh}}{\text{ch}() \hookrightarrow c}$	$\frac{\mathbf{t}' \text{ fresh}}{\langle \bar{K}, (\mathbf{t}, E[\text{spawn } e]) \parallel \bar{T} \rangle \rightarrow \langle \bar{K}, (\mathbf{t}', [e]) \parallel (\mathbf{t}, E[\text{unit}]) \parallel \bar{T} \rangle}$
SYNCTHREAD	STEPTHREAD	
$\frac{}{\langle \bar{K}, (\mathbf{t}, E[\text{sync } v]) \rangle \parallel \bar{T} \rightarrow \langle (\mathbf{t}, E, v) \parallel \bar{K}, \bar{T} \rangle}$	$\frac{e \hookrightarrow e'}{\langle \bar{K}, (\mathbf{t}, E[e]) \parallel \bar{T} \rangle \rightarrow \langle \bar{K}, (\mathbf{t}, E[e']) \parallel \bar{T} \rangle}$	
STEPTRANSACTIONALTHREAD	COMMITTRANSTHREADS	
$\frac{\bar{K} \rightsquigarrow \bar{K}'}{\langle \bar{K}, \bar{T} \rangle \rightarrow \langle \bar{K}', \bar{T} \rangle}$	$\frac{\bar{K} = (\mathbf{t}_1, E_1, \text{alwaysEvt } v_1) \parallel \dots \parallel (\mathbf{t}_n, E_n, \text{alwaysEvt } v_n) \quad \bar{T}' = \bar{T} \parallel (\mathbf{t}_1, E_1[v_1]) \parallel \dots \parallel (\mathbf{t}_n, E_n[v_n])}{\langle \bar{K}, \bar{T} \rangle \rightarrow \langle \phi, \bar{T}' \rangle}$	
STEPRUNTHREAD	NESTEDSYNC	
$\frac{e \hookrightarrow e'}{\bar{K} \parallel (\mathbf{t}, M, \mathcal{F}[e]) \rightsquigarrow \bar{K} \parallel (\mathbf{t}, M, \mathcal{F}[e'])}$	$\frac{}{\bar{K} \parallel (\mathbf{t}, M, \mathcal{F}[\text{sync } v]) \rightsquigarrow \bar{K} \parallel (\mathbf{t}, \mathcal{F} : M, v)}$	
NESTEDSYNCCOMPLETE	THENALWAYS	
$\frac{}{\bar{K} \parallel (\mathbf{t}, \mathcal{F} : M, \text{alwaysEvt } v) \rightsquigarrow \bar{K} \parallel (\mathbf{t}, M, \mathcal{F}[v])}$	$\frac{}{\bar{K} \parallel (\mathbf{t}, M, \mathcal{F}[\text{thenEvt}(\text{alwaysEvt } (v_1), v_2)]) \rightsquigarrow \bar{K} \parallel (\mathbf{t}, M, \mathcal{F}[v_2 v_1])}$	
SENDREC		
	$\frac{}{(\mathbf{t}_1, M_1, \mathcal{F}_1[\text{sendEvt}(c, v)]) \parallel (\mathbf{t}_2, M_2, \mathcal{F}_2[\text{recvEvt}(c)]) \parallel \bar{K} \rightsquigarrow (\mathbf{t}_1, M_1, \mathcal{F}_1[\text{alwaysEvt unit}]) \parallel (\mathbf{t}_2, M_2, \mathcal{F}_2[\text{alwaysEvt } v]) \parallel \bar{K}}$	
ISOLATEEVT	NESTEDISOLATEEVT	
$\frac{(\mathbf{t}_1, M_1 : \mathbf{I} : M_2, e) \notin \bar{K}}{\bar{K} \parallel (\mathbf{t}, M, \mathcal{F}[\text{isolateEvt}(v)]) \rightsquigarrow \bar{K} \parallel (\mathbf{t}, \mathbf{I} : M, \mathcal{F}[v])}$	$\frac{}{\bar{K} \parallel (\mathbf{t}, M_1 : \mathbf{I} : M_2, \mathcal{F}[\text{isolateEvt}(v)]) \rightsquigarrow \bar{K} \parallel (\mathbf{t}, M_1 : \mathbf{I} : M_2, \mathcal{F}[v])}$	
ISOLATEEVTCOMPLETE		
$\frac{}{\bar{K} \parallel (\mathbf{t}, \mathbf{I} : M, \text{alwaysEvt } v) \rightsquigarrow \bar{K} \parallel (\mathbf{t}, M, \text{alwaysEvt } v)}$		

Fig. 6. High-level semantics.

our purposes here, complex events are only built using `thenEvs`. A thread executing transactionally is represented as a triple, (τ, M, v) consisting of the thread identifier, a context stack that holds the continuation of the synchronization action, and the event. Rule `STEPTRANSACTIONALTHREAD` yields new global states based on the evaluation of expressions within transactions.

If all transactional threads in $\overline{K}$ have completed (i.e., have evaluated the event that they are synchronized upon to an `alwaysEvt(v)`), they can be removed from the transactional thread set and may resume execution as regular non-transactional threads (rule `COMMITTRANSTHREADS`). The requirement that *all* threads complete is essential for ensuring atomicity in the presence of communicating message-passing operations. The value encapsulated by the `alwaysEvt` combinator fills the context that surrounded the original `sync` operation that made the computation transactional. Recall that the context is recorded in a *context stack*. Contexts must be saved on a stack because transactions can be nested, as we describe below. When the stack contains a single entry, and the thread term is of the form `alwaysEvt(v)`, the thread is guaranteed to be no longer executing transactionally.

Transactional evaluation is defined via relation $\leadsto$ and takes place within $\mathcal{F}$ evaluation contexts; these contexts enforce atomicity of transactional execution by preventing intermediate actions from being visible to other non-transactional threads until the transaction fully completes.

The `STEPRUNTHREAD` rule allows local reductions within transactions. Rule `NESTEDSYNC` permits new transactions (initiated by a `sync` action) to be instantiated within existing ones. The current context of the outer transaction is recorded on the context transaction stack. When a transaction completes (rule `NESTEDSYNCCOMPLETE`), the saved context of the outer transaction is popped from the stack and the value yielded by the completed inner transaction is supplied to fill its hole. Since transactions can be nested, the stack is necessary to record the distinction between nested and top-level transactions, to facilitate the transition from transactional to non-transactional execution. Note that a transactional event that is synchronized upon can only successfully commit if it yields an `alwaysEvt` value – an expression that yields `neverEvt` can never be successfully synchronized.

A `thenEvt` evaluates its first event argument to an `alwaysEvt` value containing argument v_1 , and applies the function defined by its second argument (v_2) to v_1 (rule `THENALWAYS`). Two transactional threads can communicate via a `sendEvt` and `recvEvt`.

Rule `ISOLATEEVT` defines isolate evaluation. An isolate combinator given an event value enforces isolation on that event value by prohibiting concurrent evaluation of any other isolates; we record that a transactional thread is executing an isolate by explicitly marking the context stack using token **I**. Rule `NESTEDISOLATEEVT` allows a thread executing an isolate to initiate another one. When an isolate completes, the thread state reverts to an ordinary transaction (rule `ISOLATEEVTCOMPLETE`).

5 Concurrent Isolate Execution

The formulation of isolates given in the previous section enforces serializability by preventing threads executing different isolates from executing concurrently. Unfortunately, simply removing this constraint to extract greater concurrency can lead to incorrect (non-isolated) executions. Fig.7(a) depicts one such example. Here, isolate ℓ_a initiates communication with T_1 (and then T_2); similarly, isolate ℓ_b initiates a synchronous communication action with T_2 and then T_1 . The resulting global state could *not* have been produced via a serial execution of the two isolate computations – if ℓ_a were sequenced before ℓ_b (or *vice versa*) the communication with thread T_2 (or conversely, T_1) would be blocked.

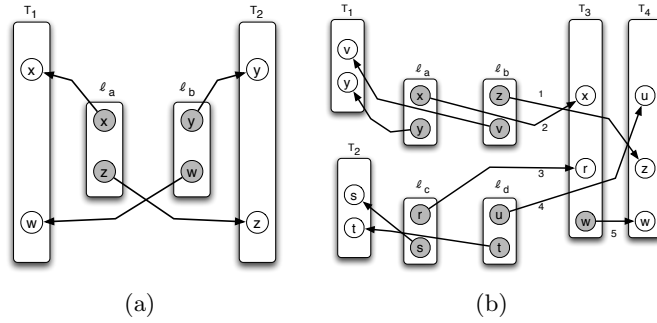


Fig. 7. Serializability violations introduced by incorrect parallel evaluation of isolates.

Even when isolate computations are serialized with respect to a single thread, care must still be taken to ensure *all* threads witness a consistent serializable view of the different isolates they communicate with. Consider the execution depicted in Fig. 7(b). Observe that T_1 witnesses an ordering in which ℓ_a executes before ℓ_b . Similarly, T_2 observes a serialization in which ℓ_c occurs before ℓ_d . Each of the isolates shown in the figure also communicate with T_3 and T_4 . Thread T_3 participates in an action with ℓ_a before an action from ℓ_c ; T_4 participates in an action with ℓ_d before ℓ_b ; the communication between T_3 and T_4 (labelled edge 5) must enforce this ordering. There are now two serializable executions induced by these different actions. From T_1 and T_4 's perspective, we need to ensure that ℓ_d, ℓ_b, ℓ_a and ℓ_c are executed atomically in that order. On the other hand, from T_2 and T_3 's perspective, the required ordering is ℓ_a, ℓ_c, ℓ_d and ℓ_b .

Clearly, there is no schedule that satisfies both evaluation order sequences, even though each thread on its own witnesses what appears to be a consistent serializable execution. The inconsistency occurs because every thread must share the same view of how isolates are serialized with respect to one another.

5.1 Capabilities

To enable scalable construction of such views, we formulate a new semantics that associates *capabilities* with threads and isolates. Capabilities are used to indicate the isolate computations a given thread may communicate with. We prevent the global serializability failure of the previous example by tracking *serialization order* globally; i.e. when an isolate discovers a potential serialization order based on how other isolates have communicated, subsequent communications that violate this order are prohibited. As such, a communication action that forces a serialization order can be viewed as a *commit point* for that isolate.

A capability is defined as a mapping between labels ℓ denoting the dynamic instances of isolate event expressions, and *tags*. When an isolate event ℓ is synchronized, a constraint is established that relates the execution of the thread executing this isolate with threads evaluating other isolates. This constraint is modeled by the tag. Intuitively, from ℓ 's perspective, the set of all isolates can be partitioned into two sets: the set L that only includes itself, and the set R that is composed of I , the set of all other evaluating isolates. To ensure a thread T 's communication with ℓ is serializable, it must be the case that either (a) T has thus far communicated only with ℓ (i.e., the sole element in set L); (b) T has thus far communicated only with isolates other than ℓ (i.e., elements in the set R); or (c) T had previously communicated with isolates in I , but now only communicates with ℓ . One possibility is prohibited: T cannot have previously communicated with ℓ , and then subsequently engaged in communication with isolates in I ; allowing T to engage in further communications with ℓ would break obvious isolation guarantees on ℓ 's execution.

Thus, suppose thread T has a capability $\ell \mapsto L$. The tag L indicates that the thread witnesses only the actions performed by ℓ , and no other isolate computation. Conversely, if T is given capability $\ell \mapsto R$, it means it has only communicated with isolates other than ℓ . Tag LR indicates that T has first communicated with ℓ and then other isolates; tag RL indicates that T has first communicated with other isolates before communicating exclusively with ℓ .

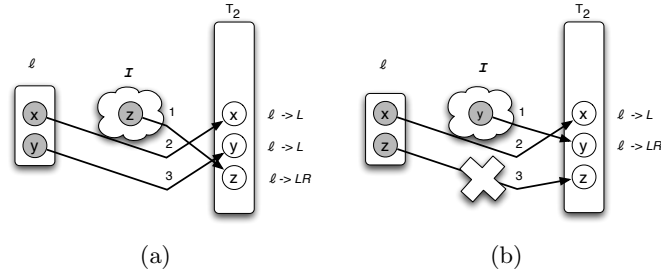


Fig. 8. Capabilities regulate communication with isolates.

Examples Consider the two examples given in Fig. 8. In both diagrams, there are two isolate expressions ℓ and I , resp. The expression I represents an abstract set of isolates. Consider the execution shown in Fig. ?? . ℓ performs two communication actions with thread T ; once these actions complete some isolate from the set I initiates a communication as well. After the communication with ℓ , T 's capability map has the entry: $[\ell \mapsto LR]$. The entry indicates that T first engaged in a communication with ℓ and then with other isolates found in I .

Now, consider the more complex example shown in Fig. 8(a)). Isolate ℓ attempts to perform two communications with thread T . Between these communications is a communication action performed by another isolate in I . The first communication by ℓ results in T acquiring a capability map containing capabilities $\ell \mapsto L$. The first capability indicates that T has only communicated with ℓ . The capability map is adjusted after the second communication via channel y . Now the capability map contains the capability $\ell \mapsto LR$. When ℓ tries to communicate via z , a serializability violation occurs: the capability $\ell \mapsto LR$ indicates that T has previously communicated with ℓ and then some other isolate in I , in that order. Allowing the communication event on z to occur would thus violate our required serializability invariant since ℓ 's actions with respect to T cannot be grouped in their entirety either before or after I 's.

5.2 Semantics

We define the semantics in Figs. 9 and 13. The semantics makes use of a new relation $\triangleright$ (see TAG ENRICHMENT) that captures the obvious ordering relationship among tags. Informally, tag ι can be enriched to tag ι' if ι imposes the same or fewer restrictions on isolate communication than ι' . Note the presence of two additional tags $\hat{L}$ and $\hat{R}$ that we haven't discussed thus far – these tags are used to build capabilities for threads executing isolate computations. If thread T executes isolate expression with label ℓ , its capability map is extended with the capability $\ell \mapsto \hat{L}$: such a capability can never be enriched since isolates, by definition, cannot communicate with other isolates. Similarly, the capability map of all other threads executing isolate computations is updated with the capability $\ell \mapsto \hat{R}$: such a capability prohibits these threads from communicating with ℓ .

Global evaluation is defined via relation $\mapsto$ that is the analog of $\rightarrow$ in the semantics given in Fig. ?? . Transactional evaluation specifically (defined by relation $\Rightarrow$) is now defined with respect to both global (Δ) and local (δ) capability maps. The global map fixes a specific serialization order for all "committed" isolates; the local map captures a thread's specific view of the isolates it has communicated with thus far prior to a commit point.

Rule CAPABILITY ENRICHMENT introduces capabilities to a thread's capability map. If the thread does not already have a tag for this isolate, it can choose one (either L or R). The thread's capability map is then updated (via auxiliary function lift) to reflect this new binding. Rule ISOLATE SERIALIZATION allows enriching a capability to a serial ordering, namely LR or RL . This rule affects the global capability map. The antecedent condition $\iota \triangleright \iota'$ leverages the structure of the relation to allow a capability that is currently L or R to become

$\begin{aligned} \ell &\in \text{Label} \\ \iota &\in \text{Tag} := \cdot \mid \hat{\mathbf{L}} \mid \hat{\mathbf{R}} \mid \mathbf{L} \mid \mathbf{R} \mid \mathbf{LR} \mid \mathbf{RL} \\ \Delta &\in \text{CapabilityMap} := \text{Label} \xrightarrow{\text{fin}} \text{Tag} \\ \delta &\in \text{LocalCapabilityMap} := \text{TID} \xrightarrow{\text{fin}} \text{CapabilityMap} \end{aligned}$	
TAG ENRICHMENT	CAPABILITY LIFTING
$\begin{aligned} \cdot &\triangleright \mathbf{L} \quad \cdot \triangleright \mathbf{R} \\ \mathbf{L} &\triangleright \mathbf{RL} \quad \mathbf{L} \triangleright \mathbf{LR} \\ \mathbf{R} &\triangleright \mathbf{LR} \quad \mathbf{R} \triangleright \mathbf{RL} \end{aligned}$	$\text{lift } \ell \ \iota \ \delta \ \mathbf{t} = \delta[\mathbf{t} \mapsto \delta(\mathbf{t})[\ell \mapsto \iota]]$
CAPABILITY ENRICHMENT	ISOLATE SERIALIZATION
$\frac{\ell \notin \text{Dom}(\delta(\mathbf{t})) \quad \iota' \in \{\mathbf{L}, \mathbf{R}\} \quad \delta' = \text{lift } \ell \ \iota' \ \delta \ \mathbf{t}}{\Delta, \delta, \overline{K} \parallel (\mathbf{t}, M, e) \Longrightarrow \Delta, \delta', \overline{K} \parallel (\mathbf{t}, M, e)}$	$\frac{\ell \notin \text{Dom}(\Delta) \quad \delta \ \mathbf{t} \ \ell = \iota \quad \iota \triangleright \iota' \quad \delta' = \text{lift } \ell \ \iota' \ \delta \ \mathbf{t}}{\Delta, \delta, \overline{K} \parallel (\mathbf{t}, M, e) \Longrightarrow \Delta[\ell \mapsto \iota'], \delta', \overline{K} \parallel (\mathbf{t}, M, e)}$
SERIALIZATION CHECK	
$\frac{\Delta(\ell) = \iota' \quad \delta \ \mathbf{t} \ \ell = \iota \quad \iota \triangleright \iota' \quad \delta' = \text{lift } \ell \ \iota' \ \delta \ \mathbf{t}}{\Delta, \delta, \overline{K} \parallel (\mathbf{t}, M, e) \Longrightarrow \Delta, \delta', \overline{K} \parallel (\mathbf{t}, M, e)}$	

Fig. 9. Capabilities and Capability Enrichment.

$\mathbf{LR}$ or $\mathbf{RL}$. Thus, this rule enforces a specific serialization order on an isolate and serves as a commit point for isolate evaluation. Since the global capability map defines a consistent serial ordering of isolates, all threads must adhere to this ordering. Rule **SERIALIZATION CHECK** thus "imports" the capability tag of an isolate found in the global state to a thread's local capability map.

Fig. 13 presents the salient rules that define transactional evaluation. Rule **SYNCTHREAD** begins transactional evaluation by initializing the local capability map of the thread performing the synchronization to the empty set, and adds a new transactional thread to $\overline{K}$. Conversely, rule **COMMITTRANSTHREADS** reverts the state from transactional execution to ordinary evaluation by "clearing" the global and local capability maps. The appendix provides a complete definition of all the rules.

Rule **ISOLATEFRESH** commences an isolate computation for a transactional event that has not yet communicated with any other isolate. A new isolate label (ℓ) is created, and the local capability map of the thread $\mathbf{t}$ performing the operation is updated to reflect this fact; δ' is the capability map that binds ℓ to $\mathbf{L}$ for $\mathbf{t}$. To prevent serializability violations, we also require that all other threads ($\mathbf{t}'$) currently executing isolate operations not communicate with this isolate. To facilitate this, we update the capability maps of these threads (δ'') to bind ℓ to $\mathbf{R}$; note that we leverage the curried definition of lift in defining the fold. Isolation is enforced by the **SENDRECV** rule shown below that prevents

SYNCTHREAD

$$\frac{}{\langle \Delta, \delta, \bar{K}, (\mathbf{t}, E[\text{sync } v]) \parallel \bar{T} \rangle \mapsto \langle \Delta, \delta[\mathbf{t} \mapsto \phi], (\mathbf{t}, E[\cdot, v]) \parallel \bar{K}, \bar{T} \rangle}$$

COMMITTRANSTHREADS

$$\frac{\bar{K} = (\mathbf{t}_1, E_1, \text{alwaysEvt } v_1) \parallel \dots \parallel (\mathbf{t}_n, E_n, \text{alwaysEvt } v_n) \quad \bar{T}' = \bar{T} \parallel (\mathbf{t}_1, E_1[v_1]) \parallel \dots \parallel (\mathbf{t}_n, E_n[v_n])}{\langle \Delta, \delta, \bar{K}, \bar{T} \rangle \mapsto \langle \phi, \phi, \phi, \bar{T}' \rangle}$$

ISOLATEFRESH

$$\frac{\ell \text{ fresh } \delta(\mathbf{t}) = \phi \quad \delta' = \text{lift } \ell \hat{\mathbf{L}} \delta \mathbf{t} \quad \bar{\mathbf{t}}' = \{\mathbf{t} \mid (\mathbf{t}, M_1 : \mathbf{I} : M_2, e') \in \bar{K} \quad \delta'' = \text{fold}(\text{lift } \ell \hat{\mathbf{R}}, \delta', \bar{\mathbf{t}}')}{\Delta, \delta, \bar{K} \parallel (\mathbf{t}, M, F[\text{isolateEvt}(v)]) \Rightarrow \Delta, \delta'', \bar{K} \parallel (\mathbf{t}, \mathbf{I} : M, F[v])}$$

ISOLATECOMM

$$\frac{\ell \text{ fresh } \delta(\mathbf{t}) \neq \phi \quad \delta' = \text{lift } \ell \mathbf{RL} \delta \mathbf{t} \quad \bar{\mathbf{t}}' = \{\mathbf{t} \mid (\mathbf{t}, M_1 : \mathbf{I} : M_2, e') \in \bar{K} \quad \delta'' = \text{fold}(\text{lift } \ell \mathbf{RL}, \delta', \bar{\mathbf{t}}')}{\Delta, \delta, \bar{K} \parallel (\mathbf{t}, M, F[\text{isolateEvt}(v)]) \Rightarrow \Delta, \delta'', \bar{K} \parallel (\mathbf{t}, \mathbf{I} : M, F[v])}$$

SENDRECV

$$\frac{\Delta' = \delta(\mathbf{t}_1) \diamond \delta(\mathbf{t}_2)}{\Delta, \delta, (\mathbf{t}_1, M_1, F_1[\text{sendEvt}(c, v)]) \parallel (\mathbf{t}_2, M_2, F_2[\text{recvEvt}(c)]) \parallel \bar{K} \Rightarrow \Delta, \delta[\mathbf{t}_1 \mapsto \Delta', \mathbf{t}_2 \mapsto \Delta'], (\mathbf{t}_1, M_1, F_1[\text{alwaysEvt unit}]) \parallel (\mathbf{t}_2, M_2, F_2[\text{alwaysEvt } v]) \parallel \bar{K}}$$

Fig. 10. Concurrent isolate evaluation using capabilities.

communication between threads that have either $\hat{\mathbf{L}}$ and $\hat{\mathbf{R}}$ bindings for the same isolate label. Thus, for a given isolate's lifetime, it cannot communicate with any other isolate either currently executing or which commences evaluation at some later point. However, isolates are still free to communicate with other threads not themselves executing isolate computations provided that such communications are permissible as defined by Fig. ??.

Rule ISOLATECOMM handles the case when a thread executing a transactional event which has previously communicated with other isolates, commences execution of a new isolate (i.e., the thread has capability bindings for other isolates in its capability map). In this case, the newly created isolate (call it I) should be serialized *after* the isolate the transaction has previously communicated with. If it were serialized before, it would imply the isolate was created prior to the communication which is obviously incorrect. The rule, therefore, chooses the appropriate serial ordering ($\mathbf{RL}$) and propagates it to all active isolates; rule ISOLATESERIALIZATION takes care of updating the global capability map with this binding when any non-isolated thread communicates with an isolate event.

Rule SENDRECV allows two threads to communicate provided their capability maps are *compatible*. Informally, two maps are compatible if the tags for the isolates they have in common are the same. More precisely, $I \diamond I'$ if $\text{Dom}(I) = \text{Dom}(I')$ and $I(\ell) \diamond I'(\ell')$ where $\hat{\mathbf{L}} \diamond \mathbf{L}$, $\hat{\mathbf{L}} \diamond \mathbf{RL}$, $\hat{\mathbf{R}} \diamond \mathbf{R}$, $\hat{\mathbf{R}} \diamond \mathbf{LR}$ and $\iota \diamond \iota$. For example, $\hat{\mathbf{L}} \diamond \mathbf{L}$ holds because a thread executing an isolate ℓ (thus having capability $\ell \mapsto \hat{\mathbf{L}}$

can certainly communicate with a thread that has previously only communicated with this isolate (that thread would therefore have capability $\ell \mapsto \mathbf{L}$). As another example, if thread T_1 had previously communicated with isolate ℓ and thread T_2 had previously communicated with isolate ℓ' , T_1 must acquire the capability for ℓ' maintained by T_2 and T_2 must acquire the capability for ℓ maintained by T_1 . Capability enrichment allows this to happen, and thus prevents T_1 from subsequently communicating with ℓ' in ways inconsistent with tag associated with T_2 's capability on ℓ' . In other words, to preserve isolation and enforce serializability, we must ensure that effects from one isolate argument does not leak into another via third-party interaction.

Notice that not none of the rules in Fig. ?? update the global capability map Δ . Consistency among the capabilities two communicating threads have is enforced by the $\diamond$ relation in rule SENDRECV that is satisfied via the capability enrichment rules.

Examples The first example shown in Fig. 11(a) is a refined version of Fig. 7(a). The isolate ℓ_a is created initially. Since there are no other isolates, it is mapped to $\hat{\mathbf{L}}$ in the capability map for the thread that executes it. Subsequently, isolate ℓ_b is created. The creation of the second isolate adds the capability $\ell_b \mapsto \hat{\mathbf{R}}$ to ℓ_a 's thread's capability map. Now, when thread $\mathbf{T}$ communicates with isolates ℓ_a it inherits ℓ_a 's bindings. No new bindings are introduced on the second communication with ℓ_a , but when $\mathbf{T}$ communicates to ℓ_b , $\mathbf{T}$'s capability for ℓ_b is lifted to $\mathbf{RL}$.

The second example, shown in Fig. 11(b), illustrates capability enrichment in the case when two threads have communicated with two separate isolates. Like the previous example, after the creation of ℓ_b , the thread executing ℓ_a has a local capability map with capabilities $\ell_a \mapsto \hat{\mathbf{L}}$ and $\ell_b \mapsto \hat{\mathbf{R}}$. When ℓ_a communicates with T_1 , T_1 acquires capabilities $\ell_a \mapsto \mathbf{L}$ and $\ell_b \mapsto \mathbf{R}$. Similarly, when T_2 communicates with isolate ℓ_b its capability map becomes $\ell_b \mapsto \mathbf{R}$. When T_1 and T_2 communicate they must coordinate to make sure they both see a consistent view of the isolates they have communicated with. Both threads agree on ℓ_a as T_2 can lift its capability for ℓ_a to $\mathbf{L}$. However, for ℓ_b , T_1 has a capability $\mathbf{R}$ and T_2 has a capability $\mathbf{L}$. In this case, either serial ordering of $\mathbf{LR}$ or $\mathbf{RL}$ is possible as the threads have never interacted before. Therefore, we simply choose a particular ordering, here $\mathbf{RL}$. After this point, neither T_1 nor T_2 can communicate with ℓ_a – the serial ordering defined by the capability map dictates that communication with ℓ_a precede all communication with ℓ_b .

6 Soundness

Our main soundness result asserts that concurrent isolate evaluation (as defined by Figs. 9 and 13) is equivalent to serial isolate evaluation (as defined by Fig. 6):

Theorem. *If*

$$\Delta, \delta, \bar{K} \parallel (\mathbf{t}, M, \mathcal{F}[\text{isolateEvt}(v)]) \Longrightarrow \dots \Longrightarrow \Delta', \delta', \bar{K}' \parallel (\mathbf{t}, \mathbf{I} : M, \mathcal{F}[\text{alwaysEvt}(v')])$$

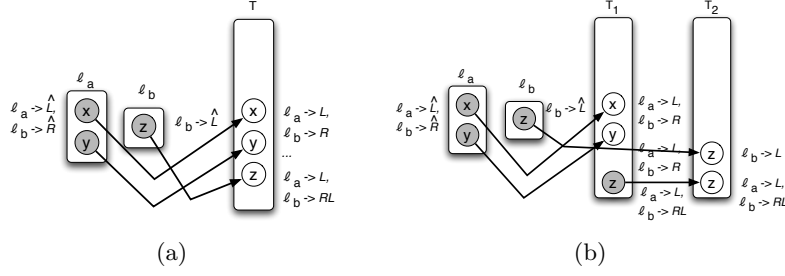


Fig. 11. Capabilities permit threads to communicate with multiple isolates, even those that are nested within one another, provided serializability invariants are preserved.

then

$$\overline{K} \parallel (\mathbf{t}, M, \mathcal{F}[\text{isolateEvt}(v)]) \rightsquigarrow \dots \rightsquigarrow \overline{K}' \parallel (\mathbf{t}, \mathbf{I} : M, \mathcal{F}[\text{alwaysEvt}(v')])$$

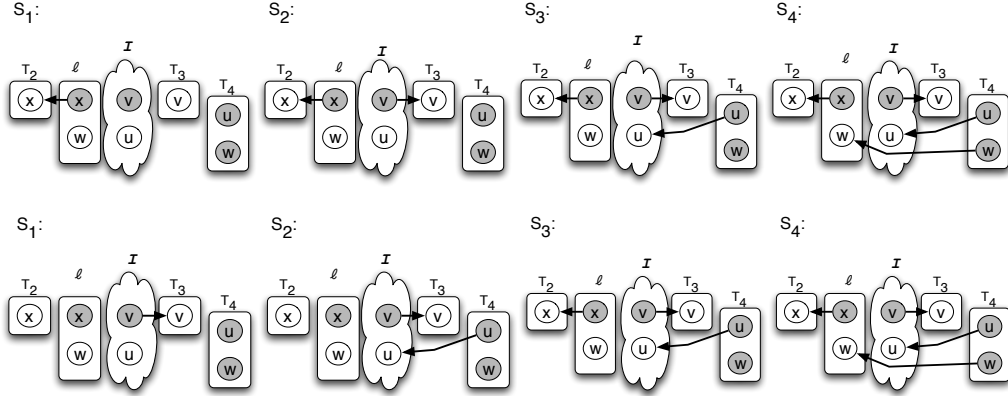


Fig. 12. Serializability of concurrent evaluation means all actions performed by the left-hand side of an isolate can be permuted before the actions performed by the right and *vice versa*.

The intuition underlying the proof for Theorem ?? is shown in Fig. 12. States S_1 through S_4 depict communication between an isolate with label ℓ and a set of other isolates I with threads T_2, T_3 and T_4 . In S_1 , isolate ℓ communicates with T_2 via channel x ; in S_2 , I communicates with T_3 via channel v ; in S_3 , there is a communication between T_4 and I ; and, finally in S_4 , T_4 establishes a communication with ℓ . Even though the isolate ℓ and the set of isolates I interleave their communication actions, their behavior is equivalent to an execution in which, for example, the I fully completes before ℓ . This alternative schedule is shown via states S'_1 through S'_4 in the figure. Here, each of the right-hand side

actions are performed before the left. Note that S_4 and S'_4 are identical. It is this permutability property on communication actions enforced by isolates that ensures our soundness result.

7 Implementation

There are three critical issues in building an efficient and feasible implementation of transactional events and isolation: (1) performing a systematic search of potential communication actions, initiating exploration of new communication pairings when an existing search yields `neverEvt`, or fails to make progress (e.g., deadlocks); (2) determining when to perform capability enrichment on local capability maps; and, (3) minimizing the cost of maintaining and updating the global map.

When a thread commences evaluation of an isolate executing within a transactional event, a new *search thread* is created (5; 6) that attempts to discharge all communication actions in that event that preserves serializability with respect to all other isolate computations. A serializability violation may manifest either because an event synchronization yields `neverEvt`, or a violation of isolation is discovered (i.e., the $\diamond$ relation fails to hold on a communication). In either case, we must abort the search thread, and revert any effects it has induced. To do this, we leverage *stabilizers* (24), a lightweight checkpointing mechanism meant to provide global state recovery for CML. Once a particular communication sequence is aborted, a new search is undertaken to explore an alternative set of communication pairings. Stabilizers provide sufficient information to synthesize a new search space from an aborted one.

We can restrict when capability enrichment on local capability maps occur to communication actions. Instead of allowing threads to lift their capabilities at arbitrary points *prior* to a communication, our implementation lifts capabilities *during* a communication. If one thread does not have a capability binding, it simply inherits one from its communication partner. If both threads contain a capability mapping then they must agree or communication is not possible. We represent capabilities using two bits of information communicated along with the message. If agreement results in a new serialization (i.e., lifting to `LR` or `RL` when neither of the communicating threads was `LR` or `RL` prior to the communication), we check the global capability map for this isolate. If there was no previous serialization of this isolate, we commit this serialization; if one existed, we check to see if the two are equal. If the new serialization does not match the old, the communication is invalid.

The global capability map needs to be consulted at most once per thread precisely at a communication. When two threads engage in a communication that must agree on a particular serialization, they consult their local capability maps. Once agreement is reached, the global capability map must be updated only if a particular serial order (i.e., `LR` or `RL`) for an isolate is required. The global capability prevents any other serialization of such isolates. Once such an

order is decided, the thread never needs to subsequently consult the global map for that isolate since the tag enrichment relation does not lift `LR` or `RL` tags.

8 Related Work and Conclusions

Our work fully integrates a transactional semantics into the CML, a concurrent extension to SML that supports synchronous message passing (17). Previous work has leveraged the atomicity guarantee of transactional semantics to add expressivity to message-passing abstractions by introducing transactional events (5; 6). These proposals allow multiple communicating actions encapsulated by transactional events to make their effects visible only once all communicating actions can succeed. However, they do not provide a means to restrict the visibility of one transactional event from another, and therefore do not guarantee serializability of transactional computations. In (6), the authors extended transactional events to support mutable references and nested synchronizations. Because this formulation of transactional events follows (5), nested transactions can always be flattened so that they execute as part of a top-level outer transaction.

In contrast, the focus of our work is a formulation of transactional events that allows programmers to achieve isolation among concurrently executing transactional events via an isolate combinator. Guaranteeing isolation requires tracking the communication of all threads that have communicated with isolates, and ensuring the communications do not violate serializability. Furthermore, nested isolates cannot be flattened, because the effects performed by a nested isolate must remain isolated from all transactional computations specified at each nesting level.

Synchronous message-passing and event abstractions, like those in CML, have also been introduced in other languages. For example, an implementation of events for Concurrent Haskell has been described in (19), for Scheme in (7) and for Caml in (4). Asynchronous message-passing has been used in languages like Erlang (2). Transactional events (5; 6) have been implemented in Concurrent Haskell and CML, respectively.

Transactional memory systems were proposed in (13). Previous work (21; 9; 3; 12; 11; 16; 22; 1; 20) has considered using software transactions to simplify reasoning about interactions between concurrently executing computations in a shared-memory system. Implementations for software transactional memory have been presented in functional languages such as Caml (18), Scheme (14) and Haskell (10). For example, the implementation of transactional events presented in (5) uses a software transactional memory extension of Concurrent Haskell (10) available in the Glasgow Haskell Compiler (GHC) (8).

There has also been other previous work that has combined inter-thread communication and software transactions. In (23) the authors extend traditional transactional memory with the ability to observe the effects of other threads at selected points. (24) describes the construction of transparent checkpoints, which can be used for transactional roll-back, in the context of CML.

A Appendix

Lemma 1. *If*

$$\begin{array}{l}
\langle \Delta, \delta, (\mathbf{t}_1, M_1, e_1) \rangle \| (\mathbf{t}_2, M_2, e_2) \| \overline{K}, \overline{T} \rangle \Longrightarrow \\
\langle \Delta, \delta, (\mathbf{t}_1, M_1, e_1) \rangle \| (\mathbf{t}_2, M_2, e'_2) \| \overline{K}, \overline{T} \rangle \Longrightarrow \\
\langle \Delta, \delta, (\mathbf{t}_1, M_1, e'_1) \rangle \| (\mathbf{t}_2, M_2, e'_2) \| \overline{K}, \overline{T} \rangle \\
\textit{then} \\
\langle \Delta, \delta, (\mathbf{t}_1, M_1, e_1) \rangle \| (\mathbf{t}_2, M_2, e_2) \| \overline{K}, \overline{T} \rangle \Longrightarrow \\
\langle \Delta, \delta, (\mathbf{t}_1, M_1, e'_1) \rangle \| (\mathbf{t}_2, M_2, e_2) \| \overline{K}, \overline{T} \rangle \Longrightarrow \\
\langle \Delta, \delta, (\mathbf{t}_1, M_1, e'_1) \rangle \| (\mathbf{t}_2, M_2, e'_2) \| \overline{K}, \overline{T} \rangle
\end{array}$$

Lemma 2. *If*

$$\begin{array}{l}
\langle \Delta, \delta, \overline{K}, (\mathbf{t}_1, e_1) || (\mathbf{t}_2, e_2) || \overline{T} \rangle \mapsto \\
\langle \Delta, \delta, \overline{K}, (\mathbf{t}_1, e'_1) || (\mathbf{t}_2, e_2) || \overline{T} \rangle \mapsto \\
\langle \Delta, \delta, \overline{K}, (\mathbf{t}_1, e'_1) || (\mathbf{t}_2, e'_2) || \overline{T} \rangle \\
\textit{then} \\
\langle \Delta, \delta, \overline{K}, (\mathbf{t}_1, e_1) || (\mathbf{t}_2, e_2) || \overline{T} \rangle \mapsto \\
\langle \Delta, \delta, \overline{K}, (\mathbf{t}_1, e_1) || (\mathbf{t}_2, e'_2) || \overline{T} \rangle \mapsto \\
\langle \Delta, \delta, \overline{K}, (\mathbf{t}_1, e'_1) || (\mathbf{t}_2, e'_2) || \overline{T} \rangle
\end{array}$$

Lemma 3. *If*

$$\begin{aligned} & \langle \Delta, \delta, \bar{K} \mid (\mathbf{t}_1, M_1, e_1) \mid (\mathbf{t}_2, M_2, e_2), \bar{T} \rangle \Longrightarrow \\ & \langle \Delta[\ell \mapsto \iota'], \delta', \bar{K} \mid (\mathbf{t}_1, M_1, e_1) \mid (\mathbf{t}_1, M_1, e_1), \bar{T} \rangle \Longrightarrow \\ & \langle \Delta[\ell \mapsto \iota', \ell' \mapsto \iota''], \delta'', \bar{K} \mid (\mathbf{t}_1, M_1, e_1) \mid (\mathbf{t}_2, M_2, e_2), \bar{T} \rangle \\ & \text{then} \\ & \langle \Delta, \delta, \bar{K} \mid (\mathbf{t}_1, M_1, e_1) \mid (\mathbf{t}_2, M_2, e_2), \bar{T} \rangle \Longrightarrow \\ & \langle \Delta[\ell' \mapsto \iota''], \delta''', \bar{K} \mid (\mathbf{t}_1, M_1, e_1) \mid (\mathbf{t}_2, M_2, e_2), \bar{T} \rangle \Longrightarrow \\ & \langle \Delta[\ell \mapsto \iota', \ell' \mapsto \iota''], \delta'', \bar{K} \mid (\mathbf{t}_1, M_1, e_1) \mid (\mathbf{t}_2, M_2, e_2), \bar{T} \rangle \end{aligned}$$

Lemma 4. *If*

$$\begin{aligned}
& \langle \Delta, \delta, (\mathbf{t}_1, M_1, e_1) \mid (\mathbf{t}_2, M_2, \mathcal{F}_2[\text{sendEvt}(ch, v)]) \mid (\mathbf{t}_3, M_3, \mathcal{F}_3[\text{recvEvt}(ch)]) \mid \overline{K}, \overline{T} \rangle \Longrightarrow \\
& \langle \Delta, \delta, (\mathbf{t}_1, M_1, e_1) \mid (\mathbf{t}_2, M_2, \mathcal{F}_2[\text{alwaysEvt unit}]) \mid (\mathbf{t}_3, M_3, \mathcal{F}_3[\text{alwaysEvt } v]) \mid \overline{K}, \overline{T} \rangle \Longrightarrow \\
& \langle \Delta, \delta, (\mathbf{t}_1, M_1, e'_1) \mid (\mathbf{t}_2, M_2, \mathcal{F}_2[\text{alwaysEvt unit}]) \mid (\mathbf{t}_3, M_3, \mathcal{F}_3[\text{alwaysEvt } v]) \mid \overline{K}, \overline{T} \rangle \\
& \text{then} \\
& \langle \Delta, \delta, (\mathbf{t}_1, M_1, e_1) \mid (\mathbf{t}_2, M_2, \mathcal{F}_2[\text{sendEvt}(ch, v)]) \mid (\mathbf{t}_3, M_3, \mathcal{F}_3[\text{recvEvt}(ch)]) \mid \overline{K}, \overline{T} \rangle \Longrightarrow \\
& \langle \Delta, \delta, (\mathbf{t}_1, M_1, e'_1) \mid (\mathbf{t}_2, M_2, \mathcal{F}_2[\text{sendEvt}(ch, v)]) \mid (\mathbf{t}_3, M_3, \mathcal{F}_3[\text{recvEvt}(ch)]) \mid \overline{K}, \overline{T} \rangle \Longrightarrow \\
& \langle \Delta, \delta, (\mathbf{t}_1, M_1, e'_1) \mid (\mathbf{t}_2, M_2, \mathcal{F}_2[\text{alwaysEvt unit}]) \mid (\mathbf{t}_3, M_3, \mathcal{F}_3[\text{alwaysEvt } v]) \mid \overline{K}, \overline{T} \rangle
\end{aligned}$$

Lemma 5. *If*

$$\begin{aligned}
& \langle \Delta, \delta, (\mathbf{t}_1, M_1, \mathcal{F}_1[\text{sendEvt}(ch_1, v_1)]) \rangle \parallel (\mathbf{t}_2, M_2, \mathcal{F}_2[\text{recvEvt}(ch_1)]) \rangle \\
& \quad \parallel (\mathbf{t}_3, M_3, \mathcal{F}_3[\text{sendEvt}(ch_2, v_2)]) \rangle \parallel (\mathbf{t}_4, M_4, \mathcal{F}_4[\text{recvEvt}(ch_2)]) \rangle \parallel \overline{K}, \overline{T} \rangle \Rightarrow \\
& \langle \Delta, \delta, (\mathbf{t}_1, M_1, \mathcal{F}_1[\text{alwaysEvt unit}]) \rangle \parallel (\mathbf{t}_2, M_2, \mathcal{F}_2[\text{alwaysEvt } v]) \rangle \\
& \quad \parallel (\mathbf{t}_3, M_3, \mathcal{F}_3[\text{sendEvt}(ch_2, v_2)]) \rangle \parallel (\mathbf{t}_4, M_4, \mathcal{F}_4[\text{recvEvt}(ch_2)]) \rangle \parallel \overline{K}, \overline{T} \rangle \Rightarrow \\
& \langle \Delta, \delta, (\mathbf{t}_1, M_1, \mathcal{F}_1[\text{alwaysEvt unit}]) \rangle \parallel (\mathbf{t}_2, M_2, \mathcal{F}_2[\text{alwaysEvt } v]) \rangle \\
& \quad \parallel (\mathbf{t}_3, M_3, \mathcal{F}_3[\text{alwaysEvt unit}]) \rangle \parallel (\mathbf{t}_4, M_4, \mathcal{F}_4[\text{alwaysEvt } v]) \rangle \parallel \overline{K}, \overline{T} \rangle \\
& \text{then} \\
& \langle \Delta, \delta, (\mathbf{t}_1, M_1, \mathcal{F}_1[\text{sendEvt}(ch_1, v_1)]) \rangle \parallel (\mathbf{t}_2, M_2, \mathcal{F}_2[\text{recvEvt}(ch_1)]) \rangle
\end{aligned}$$

SPAWN	SYNCTHREAD	STEPTHREAD
$\frac{\mathbf{t}' \text{ fresh}}{\langle \Delta, \delta, \bar{K}, (\mathbf{t}, E[\text{spawn } e]) \parallel \bar{T} \rangle \mapsto \langle \Delta, \delta, \bar{K}, (\mathbf{t}', [e]) \parallel (\mathbf{t}, E[\text{unit}]) \parallel \bar{T} \rangle}$	$\frac{}{\langle \Delta, \delta, \bar{K}, (\mathbf{t}, E[\text{sync } v]) \parallel \bar{T} \rangle \mapsto \langle \Delta, \delta[\mathbf{t} \mapsto \phi], (\mathbf{t}, E \cdot v) \parallel \bar{K}, \bar{T} \rangle}$	$\frac{e \hookrightarrow e'}{\langle \Delta, \delta, \bar{K}, (\mathbf{t}, E[e]) \parallel \bar{T} \rangle \mapsto \langle \Delta, \delta, \bar{K}, (\mathbf{t}, E[e']) \parallel \bar{T} \rangle}$
STEPTRANSACTIONALTHREAD	COMMITTRANSTHREADS	
$\frac{\Delta, \delta, \bar{K} \Longrightarrow \Delta', \delta', \bar{K}'}{\langle \Delta, \delta, \bar{K}, \bar{T} \rangle \mapsto \langle \Delta', \delta', \bar{K}', \bar{T} \rangle}$	$\frac{\bar{K} = (\mathbf{t}_1, E_1, \text{alwaysEvt } v_1) \parallel \dots \parallel (\mathbf{t}_n, E_n, \text{alwaysEvt } v_n) \quad \bar{T}' = \bar{T} \parallel (\mathbf{t}_1, E_1[v_1]) \parallel \dots \parallel (\mathbf{t}_n, E_n[v_n])}{\langle \Delta, \delta, \bar{K}, \bar{T} \rangle \mapsto \langle \phi, \phi, \phi, \bar{T}' \rangle}$	
STEPRUNTHREAD	NESTEDSYNC	
$\frac{e \hookrightarrow e'}{\Delta, \delta, \bar{K} \parallel (\mathbf{t}, \mathcal{F}[e]) \Longrightarrow \Delta, \delta, \bar{K} \parallel (\mathbf{t}, M, \mathcal{F}[e'])}$	$\frac{}{\Delta, \delta, \bar{K} \parallel (\mathbf{t}, M, \mathcal{F}[\text{sync } v]) \Longrightarrow \Delta, \delta, \bar{K} \parallel (\mathbf{t}, \mathcal{F} : M, v)}$	
NESTEDSYNCCOMPLETE	THENALWAYS	
$\frac{}{\Delta, \delta, \bar{K} \parallel (\mathbf{t}, \mathcal{F} : M, \text{alwaysEvt } v) \Longrightarrow \Delta, \delta, \bar{K} \parallel (\mathbf{t}, M, \mathcal{F}[v])}$	$\frac{}{\Delta, \delta, \bar{K} \parallel (\mathbf{t}, M, \mathcal{F}[\text{thenEvt}(\text{alwaysEvt } (v_1), v_2)]) \Longrightarrow \Delta, \delta, \bar{K} \parallel (\mathbf{t}, M, \mathcal{F}[v_2 \ v_1])}$	
ISOLATEFRESH	ISOLATECOMM	
$\frac{\ell \text{ fresh } \delta(\mathbf{t}) = \phi \ \delta' = \text{lift } \ell \ \hat{\mathbf{L}} \ \delta \ \mathbf{t} \quad \bar{\mathbf{t}}' = \{\mathbf{t} \mid (\mathbf{t}, M_1 : \mathbf{I} : M_2, e') \in \bar{K} \quad \delta'' = \text{fold}(\text{lift } \ell \ \hat{\mathbf{R}}, \delta', \bar{\mathbf{t}}')}{\Delta, \delta, \bar{K} \parallel (\mathbf{t}, M, F[\text{isolateEvt}(v)]) \Longrightarrow \Delta, \delta'', \bar{K} \parallel (\mathbf{t}, \mathbf{I} : M, F[v])}$	$\frac{\ell \text{ fresh } \delta(\mathbf{t}) \neq \phi \ \delta' = \text{lift } \ell \ \mathbf{RL} \ \delta \ \mathbf{t} \quad \bar{\mathbf{t}}' = \{\mathbf{t} \mid (\mathbf{t}, M_1 : \mathbf{I} : M_2, e') \in \bar{K} \quad \delta'' = \text{fold}(\text{lift } \ell \ \mathbf{RL}, \delta', \bar{\mathbf{t}}')}{\Delta, \delta, \bar{K} \parallel (\mathbf{t}, M, F[\text{isolateEvt}(v)]) \Longrightarrow \Delta, \delta'', \bar{K} \parallel (\mathbf{t}, \mathbf{I} : M, F[v])}$	
SENDRECV		
$\frac{\Delta' = \delta(\mathbf{t}_1) \diamond \delta(\mathbf{t}_2)}{\Delta, \delta, (\mathbf{t}_1, M_1, F_1[\text{sendEvt}(c, v)]) \parallel (\mathbf{t}_2, M_2, F_2[\text{recvEvt}(c)]) \parallel \bar{K} \Longrightarrow \Delta, \delta[\mathbf{t}_1 \mapsto \Delta', \mathbf{t}_2 \mapsto \Delta'], (\mathbf{t}_1, M_1, F_1[\text{alwaysEvt unit}]) \parallel (\mathbf{t}_2, M_2, F_2[\text{alwaysEvt } v]) \parallel \bar{K}}$		
NESTEDISOLATEEVT	ISOLATEEVTCOMPLETE	
$\frac{}{\Delta, \delta, \bar{K} \parallel (\mathbf{t}, M_1 : \mathbf{I} : M_2, F[\text{isolateEvt}(v)]) \Longrightarrow \Delta, \delta, \bar{K} \parallel (\mathbf{t}, M_1 : \mathbf{I} : M_2, F[v])}$	$\frac{}{\Delta, \delta, \bar{K} \parallel (\mathbf{t}, \mathbf{I} : M, \text{alwaysEvt } v) \Longrightarrow \Delta, \delta, \bar{K} \parallel (\mathbf{t}, M, \text{alwaysEvt } v)}$	

Fig. 13. Semantics for concurrent isolate evaluation using capabilities.

$$\begin{aligned}
& ||(\mathbf{t}_3, M_3, \mathcal{F}_3[\text{sendEvt}(ch_2, v_2)])||(\mathbf{t}_4, M_4, \mathcal{F}_4[\text{recvEvt}(ch_2)])||\overline{K}, \overline{T}\rangle \implies \\
& \langle \Delta, \delta, (\mathbf{t}_1, M_1, \mathcal{F}_1[\text{sendEvt}(ch_1, v_1)])||(\mathbf{t}_2, M_2, \mathcal{F}_2[\text{recvEvt}(ch_1)]) \\
& ||(\mathbf{t}_3, M_3, \mathcal{F}_3[\text{alwaysEvtunit}])||(\mathbf{t}_4, M_4, \mathcal{F}_4[\text{alwaysEvt}v])||\overline{K}, \overline{T}\rangle \implies \\
& \langle \Delta, \delta, (\mathbf{t}_1, M_1, \mathcal{F}_1[\text{alwaysEvt unit}])||(\mathbf{t}_2, M_2, \mathcal{F}_2[\text{alwaysEvt } v]) \\
& ||(\mathbf{t}_3, M_3, \mathcal{F}_3[\text{alwaysEvtunit}])||(\mathbf{t}_4, M_4, \mathcal{F}_4[\text{alwaysEvt}v])||\overline{K}, \overline{T}\rangle
\end{aligned}$$

Lemma 6. *If*

$$\langle \Delta, \delta, \overline{K}, \overline{T} \rangle \mapsto \langle \Delta', \delta', \overline{K}', \overline{T} \rangle \mapsto \langle \Delta', \delta', \overline{K}', \overline{T}' \rangle$$

then

$$\langle \Delta, \delta, \overline{K}, \overline{T} \rangle \mapsto \langle \Delta, \delta, \overline{K}, \overline{T}' \rangle \mapsto \langle \Delta, \delta, \overline{K}', \overline{T}' \rangle$$

Lemma 7. *If*

$$\langle \Delta, \delta, \overline{K}, \overline{T} \rangle \mapsto \langle \phi, \phi, \phi, \overline{T} \parallel \overline{T}' \rangle \mapsto \langle \phi, \phi, \phi, \overline{T}'' \parallel \overline{T}' \rangle$$

then

$$\langle \Delta, \delta, \overline{K}, \overline{T} \rangle \mapsto \langle \Delta, \delta, \overline{K}, \overline{T}'' \rangle \mapsto \langle \phi, \phi, \phi, \overline{T}'' \parallel \overline{T}' \rangle$$

Lemma 8. *If*

$$\langle \Delta, \delta, \overline{K}, \overline{T} \parallel (\mathbf{t}, E[\text{syncv}]) \rangle \mapsto \langle \Delta, \delta, \overline{K} \parallel (\mathbf{t}, E, v), \overline{T} \rangle \mapsto \langle \Delta, \delta, \overline{K} \parallel (\mathbf{t}, E, v), \overline{T}' \rangle$$

then

$$\langle \Delta, \delta, \overline{K}, \overline{T} \parallel (\mathbf{t}, E[\text{syncv}]) \rangle \mapsto \langle \Delta, \delta, \overline{K}, \overline{T}' \parallel (\mathbf{t}, E[\text{syncv}]) \rangle \mapsto \langle \Delta, \delta, \overline{K} \parallel (\mathbf{t}, E, v), \overline{T}' \rangle$$

Theorem 1. (Soundness of Transactional Evaluation)

If

$$\Delta, \delta, \overline{K} \parallel (\mathbf{t}, M, \mathcal{F}[\text{isolateEvt}(v)]) \implies \dots \implies \Delta', \delta', \overline{K}' \parallel (\mathbf{t}, \mathbf{I} : M, \mathcal{F}[\text{alwaysEvt}(v')])$$

then

$$\overline{K} \parallel (\mathbf{t}, M, \mathcal{F}[\text{isolateEvt}(v)]) \rightsquigarrow \dots \rightsquigarrow \overline{K}' \parallel (\mathbf{t}, \mathbf{I} : M, \mathcal{F}[\text{alwaysEvt}(v')])$$

Proof Theorem 1.

Base Case

Inductive Case

Theorem 2. (Soundness) *If*

$$\langle \Delta, \delta, \overline{K} \parallel (\mathbf{t}, M, \mathcal{F}[\text{isolateEvt}(v)]), \overline{T} \rangle \mapsto \dots \mapsto \langle \Delta', \delta', \overline{K}' \parallel (\mathbf{t}, \mathbf{I} : M, \mathcal{F}[\text{alwaysEvt}(v')]), \overline{T}' \rangle$$

then

$$\langle \overline{K} \parallel (\mathbf{t}, M, \mathcal{F}[\text{isolateEvt}(v)]), \overline{T} \rangle \rightarrow \dots \rightarrow \langle \overline{K}' \parallel (\mathbf{t}, \mathbf{I} : M, \mathcal{F}[\text{alwaysEvt}(v')]), \overline{T}' \rangle$$

Proof Theorem 2.

Base Case

Assume:

$$\langle \Delta, \delta, \overline{K} \parallel (\mathbf{t}, M, \mathcal{F}[\text{isolateEvt}(v)]), \overline{T} \rangle \mapsto \langle \Delta', \delta', \overline{K}' \parallel (\mathbf{t}, \mathbf{I} : M, \mathcal{F}[\text{alwaysEvt}(v')]), \overline{T} \rangle$$

By **Theorem 1.**

$$\langle \overline{K} \parallel (\mathbf{t}, M, \mathcal{F}[\text{isolateEvt}(v)]), \overline{T} \rangle \rightarrow \langle \overline{K}' \parallel (\mathbf{t}, \mathbf{I} : M, \mathcal{F}[\text{alwaysEvt}(v')]), \overline{T} \rangle$$

□

Inductive Case

Assume **Theorem 2.** holds for evaluation sequences of length n :

We know:

$$\langle \Delta, \delta, \bar{K} \parallel (\mathbf{t}, M, \mathcal{F}[\text{isolateEvt}(v)]), \bar{T} \rangle \mapsto \dots \mapsto \langle \Delta'', \delta'', \bar{K}'', \bar{T}'' \rangle$$

If:

$$\langle \Delta'', \delta'', \bar{K}'', \bar{T}'' \rangle \mapsto \langle \Delta', \delta', \bar{K}' \parallel (\mathbf{t}, \mathbf{I} : M, \mathcal{F}[\text{alwaysEvt}(v')]), \bar{T}' \rangle$$

Need to show:

$$\langle \bar{K}'', \bar{T}'' \rangle \rightarrow \langle \bar{K}' \parallel (\mathbf{t}, \mathbf{I} : M, \mathcal{F}[\text{alwaysEvt}(v')]), \bar{T}' \rangle$$

Case `spawn`:

By definition of `spawn`:

$$\bar{T}'' = \bar{T}''' \parallel (\mathbf{t}, E[\text{spawn } e])$$

$$\bar{T}' = (\mathbf{t}', [e]) \parallel (\mathbf{t}, E[\text{unit}]) \parallel \bar{T}'''$$

$$\bar{K}'' = \bar{K}' \parallel (\mathbf{t}, \mathbf{I} : M, \mathcal{F}[\text{alwaysEvt}(v')])$$

Implies:

$$\langle \bar{K}'', \bar{T}'' \rangle \rightarrow \langle \bar{K}' \parallel (\mathbf{t}, \mathbf{I} : M, \mathcal{F}[\text{alwaysEvt}(v')]), \bar{T}' \rangle$$

Case `syncThread`:

By definition of `syncThread`:

$$e \hookrightarrow e'$$

$$\bar{T}'' = \text{threadt}E[\text{sync } v] \parallel \bar{T}'$$

$$\bar{K}'' = \bar{K}''' \parallel \parallel (\mathbf{t}, \mathbf{I} : M, \mathcal{F}[\text{alwaysEvt}(v')])$$

$$\bar{K}' = \bar{K}''' \parallel (\mathbf{t}, E \vdash, v)$$

Implies:

$$\langle \bar{K}'', \bar{T}'' \rangle \rightarrow \langle \bar{K}' \parallel (\mathbf{t}, \mathbf{I} : M, \mathcal{F}[\text{alwaysEvt}(v')]), \bar{T}' \rangle$$

Case `stepThread`:

By definition of `stepThread`:

$$\bar{T}'' = (\mathbf{t}, E[e]) \parallel \bar{T}'''$$

$$\bar{T}' = (\mathbf{t}, E[e']) \parallel \bar{T}'''$$

$$\bar{K}'' = \bar{K}' \parallel (\mathbf{t}, \mathbf{I} : M, \mathcal{F}[\text{alwaysEvt}(v')])$$

Implies:

$$\langle \bar{K}'', \bar{T}'' \rangle \rightarrow \langle \bar{K}' \parallel (\mathbf{t}, \mathbf{I} : M, \mathcal{F}[\text{alwaysEvt}(v')]), \bar{T}' \rangle$$

Case `stepTransactionalThread`:

By definition of `stepTransactionalThread`:

$$\Delta'', \delta'', \bar{K}'' \Longrightarrow \Delta', \delta', \bar{K}' \parallel (\mathbf{t}, \mathbf{I} : M, \mathcal{F}[\text{alwaysEvt}(v')])$$

By **Theorem 1.**:

$$\bar{K}'' \rightarrow \bar{K}' \parallel (\mathbf{t}, \mathbf{I} : M, \mathcal{F}[\text{alwaysEvt}(v')])$$

Case `commitTransThread`:

By definition of `commitTransThread` this reduction cannot be applied.

□

Bibliography

- [1] Ali-Reza Adl-Tabatabai, Brian T. Lewis, Vijay Menon, Brian R. Murphy, Bratin Saha, and Tatiana Shpeisman. Compiler and Runtime Support for Efficient Software Transactional Memory. In *PLDI*, pages 26–37, 2006.
- [2] Joe Armstrong, Robert Virding, Claes Wikstrom, and Mike Williams. *Concurrent Programming in Erlang*. Prentice-Hall, 2nd edition, 1996.
- [3] Colin Blundell, E. Christopher Lewis, and Milo Martin. Subtleties of Transactional Memory Atomicity Semantics. *IEEE Computer Architecture Letters*, 5(2), 2006.
- [4] The Objective Caml System Release 3.10, Event Module, 2007.
- [5] Kevin Donnelly and Matthew Fluet. Transactional Events. *The Journal of Functional Programming*, pages 649–706, 2008.
- [6] Laura Effinger-Dean, Matthew Kehrt, and Dan Grossman. Transactional Events for ML. In *ICFP*, pages 103–114, 2008.
- [7] Matthew Flatt and Robert Bruce Findler. Kill-safe Synchronization Abstractions. In *PLDI*, pages 47–58, 2004.
- [8] Glasgow Haskell Compiler. <http://www.haskell.org/ghc>.
- [9] Tim Harris and Keir Fraser. Language support for Lightweight Transactions. In *OOPSLA*, pages 388–402, 2003.
- [10] Tim Harris, Simon Marlow, Simon Peyton Jones, , and Maurice Herlihy. Composable Memory Transactions. In *PPoPP*, pages 48–60, 2005.
- [11] Maurice Herlihy, Victor Luchangco, and Mark Moir. A Flexible Framework for Implementing Software Transactional Memory. In *OOPSLA*, pages 253–262, 2006.
- [12] Maurice Herlihy, Victor Luchangco, Mark Moir, and III William Scherer. Software Transactional Memory for Dynamic-Sized Data Structures. In *PODC*, pages 92–101, 2003.
- [13] Maurice Herlihy and J. Eliot B. Moss. Transactional Memory: Architectural Support for Lock-Free Data Structures. In *ISCA*, pages 289–300, 1993.
- [14] Aaron Kimball and Dan Grossman. Software Transactions Meet First-Class Continuations. In *Scheme Workshop*, pages 47–58, 2007.
- [15] James Larus and Ravi Rajwar. *Transactional Memory*. Morgan & Claypool, 2007.
- [16] V.J. Marathe, William N. Scherer III, and Michael L. Scott. Design Tradeoffs in Modern Software Transactional Memory Systems. In *LCR*, pages 1–7, 2004.
- [17] John Reppy. *Concurrent Programming in ML*. Cambridge University Press, 1999.
- [18] Michael F. Ringenburt and Dan Grossman. AtomCaml: First-class atomicity via Rollback. In *ICFP*, pages 92–104, 2005.
- [19] George Russell. Events in Haskell, and How to Implement Them. In *ICFP*, pages 157–168, 2001.
- [20] Bratin Saha, Ali-Reza Adl-Tabatabai, Richard L. Hudson, Chi Cao Minh, and Benjamin Hertzberg. McRT-STM: a High-Performance Software Transactional Memory system for a Multi-Core Runtime. In *PPoPP*, pages 187–197, 2006.
- [21] Nir Shavit and Dan Touitou. Software Transactional Memory. In *PODC*, pages 204–213, 1995.
- [22] Tatiana Shpeisman, Vijay Menon, Ali-Reza Adl-Tabatabai, Steven Balensiefer, Dan Grossman, Richard L. Hudson, Katherine F. Moore, and Bratin Saha. Enforcing Isolation and Ordering in STM. In *PLDI*, pages 78–88, 2007.
- [23] Yannis Smaragdakis, Anthony Kay, Reimer Behrendts, and Michal Young. Transactions with Isolation and Cooperation. In *OOPSLA*, pages 191–210, 2007.

- [24] Lukasz Ziarek, Philip Schatz, and Suresh Jagannathan. Stabilizers: A Modular Checkpointing Abstraction for Concurrent Functional Programs. In *ICFP*, pages 136–147, 2006.